Carlos Coronel • Steven Morris

Database Systems

Design, Implementation, & Management

National Economics University
Faculty of Data Science & Artificial Intelligence

Database System Design, Implementation And Management Course – Introduction to Structured Query Language (SQL)

- Instructor: Duc Minh Vu (FDA – SLSCM Lab)
- Email: minhvd [at] neu.edu.vn

Learning Objectives

- **7-1** Retrieve specified columns of data from a database
- **7-2** Join multiple tables in a single SQL query
- **7-3** Restrict data retrievals to rows that match complex criteria
- **7-4** Aggregate data across groups of rows
- **7-5** Create subqueries to preprocess data for inclusion in other queries
- **7-6** Identify and use a variety of SQL functions for string, numeric, and date manipulation
- **7-7** Explain the key principles in crafting a SELECT query

7-1 SQL Basics

7-1 SQL Basics

- It is a *data manipulation language (DML)*.
  - SQL includes commands to insert, update, delete, and retrieve data within the database tables.
- It is a *data definition language (DDL)*.
  - SQL includes commands to create database objects such as tables, indexes, and views, as well as commands to define access rights to those database objects.
- It is a *transaction control language (TCL)*.
  - The DML commands in SQL are executed within the context of a **transaction**, which is a logical unit of work composed of one or more SQL statements, as defined by business rules.
- It is a *data control language (DCL)*.
  - Data control commands are used to control access to data object

Table 7.1 SQL Data Manipulation Commands

Command, Option, or Operator	Description	Covered
SELECT	**Selects attributes from rows in one or more tables or views**	**Chapter 7**
FROM	Specifies the tables from which data should be retrieved	Chapter 7
WHERE	Restricts the selection of rows based on a conditional expression	Chapter 7
GROUP BY	Groups the selected rows based on one or more attributes	Chapter 7
HAVING	Restricts the selection of grouped rows based on a condition	Chapter 7
ORDER BY	Orders the selected rows based on one or more attributes	Chapter 7
INSERT	**Inserts row(s) into a table**	**Chapter 8**
UPDATE	**Modifies an attribute's values in one or more rows of a table**	**Chapter 8**
DELETE	**Deletes one or more rows from a table**	**Chapter 8**
Comparison operators		**Chapter 7**
=, <, >, <=, >=, <>, !=	Used in conditional expressions	Chapter 7
Logical operators		**Chapter 7**
AND/OR/NOT	Used in conditional expressions	Chapter 7

Special operators	**Used in conditional expressions**	**Chapter 7**
BETWEEN	Checks whether an attribute value is within a range	Chapter 7
IN	Checks whether an attribute value matches any value within a value list	Chapter 7
LIKE	Checks whether an attribute value matches a given string pattern	Chapter 7
IS NULL	Checks whether an attribute value is null	Chapter 7
EXISTS	Checks whether a subquery returns any rows	Chapter 7
DISTINCT	Limits values to unique values	Chapter 7
Aggregate functions	**Used with SELECT to return mathematical summaries on columns**	**Chapter 7**
COUNT	Returns the number of rows with non-null values for a given column	Chapter 7
MIN	Returns the minimum attribute value found in a given column	Chapter 7
MAX	Returns the maximum attribute value found in a given column	Chapter 7
SUM	Returns the sum of all values for a given column	Chapter 7
AVG	Returns the average of all values for a given column	Chapter 7

Table 7.2 SQL Data Definition Commands

Command or Option	Description	Covered
CREATE SCHEMA AUTHORIZATION	**Creates a database schema**	**Chapter 8**
CREATE TABLE	**Creates a new table in the user's database schema**	**Chapter 8**
NOT NULL	Ensures that a column will not have null values	Chapter 8
UNIQUE	Ensures that a column will not have duplicate values	Chapter 8
PRIMARY KEY	Defines a primary key for a table	Chapter 8
FOREIGN KEY	Defines a foreign key for a table	Chapter 8
DEFAULT	Defines a default value for a column (when no value is given)	Chapter 8
CHECK	Validates data in an attribute	Chapter 8
CREATE INDEX	**Creates an index for a table**	**Chapter 8**
CREATE VIEW	**Creates a dynamic subset of rows and columns from one or more tables**	**Chapter 8**
ALTER TABLE	**Modifies a table's definition (adds, modifies, or deletes attributes or constraints)**	**Chapter 8**
CREATE TABLE AS	**Creates a new table based on a query in the user's database schema**	**Chapter 8**
DROP TABLE	**Permanently deletes a table (and its data)**	**Chapter 8**
DROP INDEX	**Permanently deletes an index**	**Chapter 8**
DROP VIEW	**Permanently deletes a view**	**Chapter 8**

Table 7.3 Other SQL Commands

Command or Option	Description	Covered
Transaction Control Language		
COMMIT	Permanently saves data changes	Chapter 8
ROLLBACK	Restores data to its original values	Chapter 8
Data Control Language		
GRANT	Gives a user permission to take a system action or access a data object	Chapter 16
REVOKE	Removes a previously granted permission from a user	Chapter 16

7-1a Data Types

- The ANSI/ISO SQL standard defines many data types.
- A data type is a specification about the kinds of data that can be stored in an attribute.
- *Character data* is composed of any printable characters such as alphabetic values, digits, punctuation, and special characters.
- *Numeric data* is composed of digits, such that the data has a specific numeric value.
- *Date data* is composed of date and, occasionally, time values.

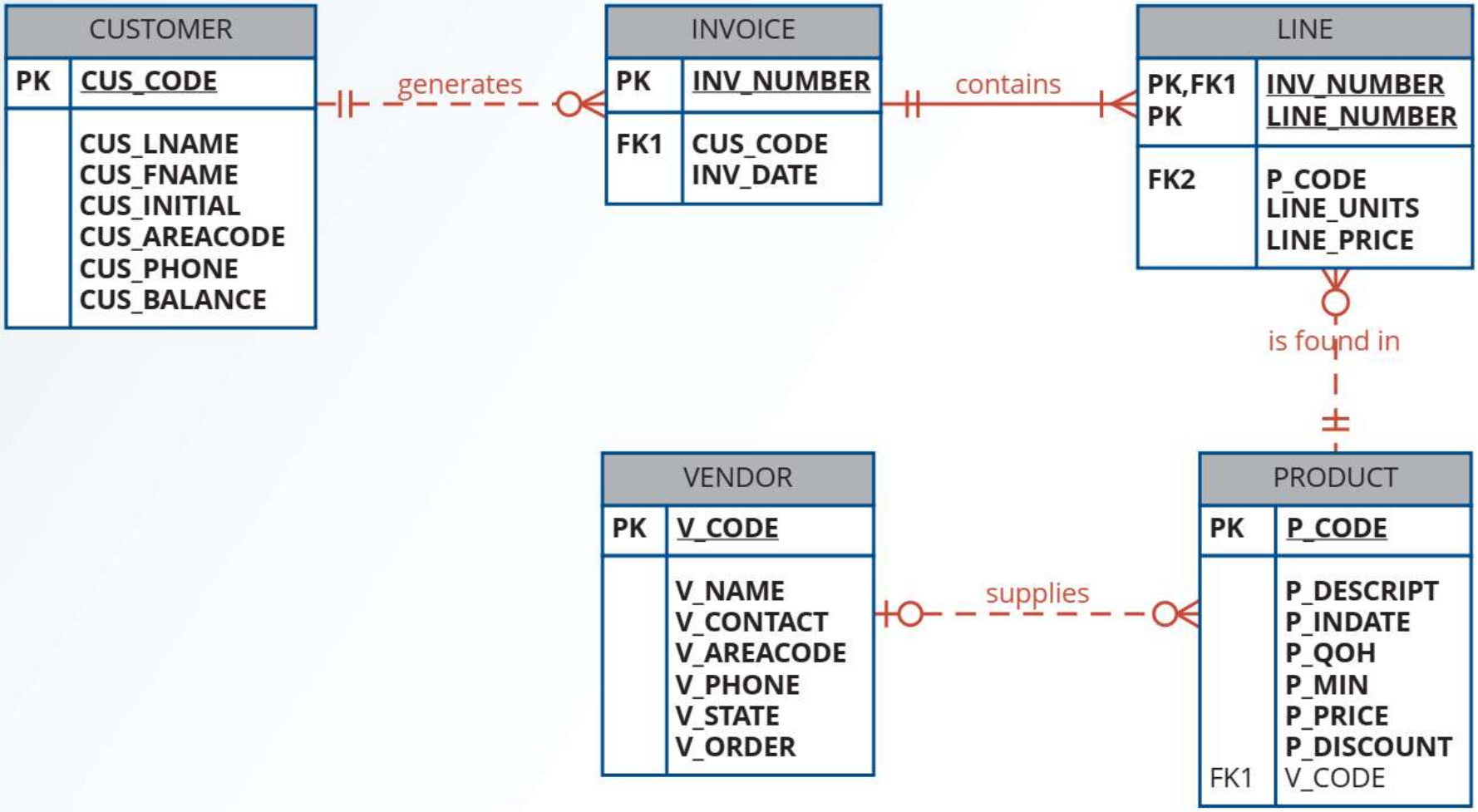
7-1b SQL Queries

- in the SQL environment, the word *query* covers both questions and actions.
- SQL queries are used to answer questions such as these: “What products currently held in inventory are priced over $100, and what is the quantity on hand for each of those products?”
- SQL queries are used to perform actions such as adding or deleting table rows or changing attribute values within tables.
- Data retrieval is done in SQL using a SELECT query.
  - RDBMS returns a set of one or more rows that have the same characteristics as a relational table
  - *set-oriented* commands

7-1c The Database Model

- CUSTOMER, INVOICE, LINE, PRODUCT, and VENDOR
- A customer may generate many invoices. Each invoice is generated by one customer.
- An invoice contains one or more invoice lines. Each invoice line is associated with one invoice.
- Each invoice line references one product. A product may be found in many invoice lines. (You can sell more than one hammer to more than one customer.)
- A vendor may supply many products. Some vendors do not yet supply products. For example, a vendor list may include potential vendors.
- If a product is vendor-supplied, it is supplied by only a single vendor.
- Some products are not supplied by a vendor. For example, some products may be produced in-house or bought on the open market.

Figure 7.1 The Database Model



7-2 Basic SELECT Queries

- SELECT—specifies the attributes to be returned by the query
- FROM—specifies the table(s) from which the data will be retrieved
- WHERE—filters the rows of data based on provided criteria
- GROUP BY—groups the rows of data into collections based on sharing the same values in one or more attributes
- HAVING—filters the groups formed in the GROUP BY clause based on provided criteria
- ORDER BY—sorts the final query result rows in ascending or descending order based on the values of one or more attributes.

7-3 SELECT Statement Options

7-3 SELECT Statement Options

- The SELECT query specifies the columns to be retrieved as a column list.
- The syntax for a basic SELECT query that retrieves data from a table is:

  SELECT *columnlist*

  FROM *tablelist*;

- The *columnlist* represents one or more attributes, separated by commas.
- If the programmer wants all of the columns to be returned, then the asterisk (*) wildcard can be used.

  SELECT *

  FROM PRODUCT;

wildcard character
A symbol that can be used as a general substitute for: (1) all columns in a table (*) when used in an attribute list of a SELECT statement or (2) zero or more characters in a SQL LIKE clause condition (% and _).

Figure 7.2 SELECT an Entire Table

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-21	8	5	109.99	0.00	25595
13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-21	32	15	14.99	0.05	21344
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-21	18	12	17.49	0.00	21344
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-22	15	8	39.95	0.00	23119
1558-QVV1	Hrd. cloth, 1/2-in., 3x50	15-Jan-22	23	5	43.99	0.00	23119
2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-21	8	5	109.92	0.05	24288
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-21	6	5	99.87	0.05	24288
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-22	12	5	38.95	0.05	25595
23109-HB	Claw hammer	20-Jan-22	23	10	9.95	0.10	21225
23114-AA	Sledge hammer, 12 lb.	02-Jan-22	8	5	14.40	0.05	
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-21	43	20	4.99	0.00	21344
89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-22	11	5	256.99	0.05	24288
PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-22	188	75	5.87	0.00	
SM-18277	1.25-in. metal screw, 25	01-Mar-22	172	75			
SW-23116	2.5-in. wd. screw, 50	24-Feb-22	237	100			
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-22	18	5			

```
SELECT    *
FROM      PRODUCT;
```

Figure 7.3 SELECT with a Column List

P_CODE	P_DESCRIPT	P_PRICE	P_QOH
11QER/31	Power painter, 15 psi., 3-nozzle	109.99	8
13-Q2/P2	7.25-in. pwr. saw blade	14.99	32
14-Q1/L3	9.00-in. pwr. saw blade	17.49	18
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	39.95	15
1558-QVV1	Hrd. cloth, 1/2-in., 3x50	43.99	23
2232/QTY	B&D jigsaw, 12-in. blade	109.92	8
2232/QWE	B&D jigsaw, 8-in. blade	99.87	6
2238/QPD	B&D cordless drill, 1/2-in.	38.95	12
23109-HB	Claw hammer	9.95	23
23114-AA	Sledge hammer, 12 lb.	14.40	8
54778-2T	Rat-tail file, 1/8-in. fine	4.99	43
89-WRE-Q	Hicut chain saw, 16 in.	256.99	11
PVC23DRT	PVC pipe, 3.5-in., 8-ft	5.87	188
SM-18277	1.2		
SW-23116	2.5		
WR3/TT3	St		

```
SELECT    P_CODE, P_DESCRIPT, P_PRICE, P_QOH
FROM      PRODUCT;
```

7-3a Using Column Aliases

- **Alias** An alternative name for a column or table in a SQL statement.
- Not all columns in a query must use an alias
- AS is optional, but recommended
- Aliases that contain a space must be inside a delimiter (quotes)

```
SELECT      P_CODE, P_DESCRIPT AS DESCRIPTION, P_PRICE AS "Unit Price",
            P_QOH QTY
FROM        PRODUCT;
```

Figure 7.4 SELECT with Column Aliases

P_CODE	DESCRIPTION	Unit Price	QTY
11QER/31	Power painter, 15 psi., 3-nozzle	109.99	8
13-Q2/P2	7.25-in. pwr. saw blade	14.99	32
14-Q1/L3	9.00-in. pwr. saw blade	17.49	18
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	39.95	15
1558-QVV1	Hrd. cloth, 1/2-in., 3x50	43.99	23
2232/QTY	B&D jigsaw, 12-in. blade	109.92	8
2232/QWE	B&D jigsaw, 8-in. blade	99.87	6
2238/QPD	B&D cordless drill, 1/2-in.	38.95	12
23109-HB	Claw hammer	9.95	23
23114-AA	Sledge hammer, 12 lb.	14.40	8
54778-2T	Rat-tail file, 1/8-in. fine	4.99	43
89-WRE-Q	Hicut chain saw, 16 in.	256.99	11
PVC23DRT	PVC pipe, 3.5-in., 8-ft	5.87	188
SM-18277	1.25-in. metal screw, 25	6.99	172
SW-23116	2.5-in.		
WR3/TT3	Steel r		

```
SELECT    P_CODE, P_DESCRIPT AS DESCRIPTION, P_PRICE AS "Unit Price",
          P_QOH QTY
FROM      PRODUCT;
```

Note

Using delimiters with column aliases even when the alias does not contain a space can serve other purposes. In some DBMSs, if the column alias is not placed inside a delimiter, it is automatically converted to uppercase letters. In those cases, using the delimiter allows the programmer to control the capitalization of the column alias. Using delimiters also allows a column alias to contain a special character, such as a "+", or a SQL keyword, such as "SELECT." In general, using special characters and SQL keywords in column aliases is discouraged, but it is possible.

Note

MySQL uses a special delimiter, the back tick "`" (usually found to the left of the number 1 on a standard keyboard) as a delimiter for column aliases if you want to refer to that alias elsewhere within the query, such as the ORDER BY clause covered later in this chapter.

7-3b Using Computed Columns

- A computed column (also called a calculated column) represents a derived attribute.
- Suppose that you want to determine the total value of each product currently held in inventory
- Require multiplying each product's quantity on hand by its current price

```
SELECT    P_DESCRIPT, P_QOH, P_PRICE, P_QOH * P_PRICE
FROM      PRODUCT;
```

Figure 7.5 SELECT Statement with a Computed Column

P_DESCRIPT	P_QOH	P_PRICE	Expr1
Power painter, 15 psi., 3-nozzle	8	109.99	879.92
7.25-in. pwr. saw blade	32	14.99	479.68
9.00-in. pwr. saw blade	18	17.49	314.82
Hrd. cloth, 1/4-in., 2x50	15	39.95	599.25
Hrd. cloth, 1/2-in., 3x50	23	43.99	1011.77
B&D jigsaw, 12-in. blade	8	109.92	879.36
B&D jigsaw, 8-in. blade	6	99.87	599.22
B&D cordless drill, 1/2-in.	12	38.95	467.40
Claw hammer	23	9.95	228.85
Sledge hammer, 12 lb.	8	14.40	115.20
Rat-tail file, 1/8-in. fine	43	4.99	214.57
Hicut chain saw, 16 in.	11	256.99	2826.89
PVC pipe, 3.5-in., 8-ft	188	5.87	1103.56
1.25-in. metal screw, 25	172	6.99	1202.28

```
SELECT    P_DESCRIPT, P_QOH, P_PRICE, P_QOH * P_PRICE
FROM      PRODUCT;
```

Figure 7.6 SELECT Statement with a Computed Column and an Alias

P_DESCRIPT	P_QOH	P_PRICE	TOTVALUE
Power painter, 15 psi., 3-nozzle	8	109.99	879.92
7.25-in. pwr. saw blade	32	14.99	479.68
9.00-in. pwr. saw blade	18	17.49	314.82
Hrd. cloth, 1/4-in., 2x50	15	39.95	599.25
Hrd. cloth, 1/2-in., 3x50	23	43.99	1011.77
B&D jigsaw, 12-in. blade	8	109.92	879.36
B&D jigsaw, 8-in. blade	6	99.87	599.22
B&D cordless drill, 1/2-in.	12	38.95	467.40
Claw hammer	23	9.95	228.85
Sledge hammer, 12 lb.	8	14.40	115.20
Rat-tail file, 1/8-in. fine	43	4.99	214.57
Hicut chain saw, 16 in.	11	256.99	2826.89
PVC pipe, 3.5-in., 8-ft	188	5.87	1103.56

```
SELECT    P_DESCRIPT, P_QOH, P_PRICE, P_QOH * P_PRICE AS TOTVALUE
FROM      PRODUCT;
```

7-3c Arithmetic Operators: The Rule of Precedence

- **rules of precedence** Basic algebraic rules that specify the order in which operations are performed

Table 7.4 The Arithmetic Operators

Operator	Description
+	Add
–	Subtract
*	Multiply
/	Divide
^	Raise to the power of (some applications use ** instead of ^)

7-3d Date Arithmetic

- a date is stored as a day number, that is, the number of days that have passed since some defined point in history
- it is possible to perform date arithmetic in a query

```
SELECT    P_CODE, P_INDATE, P_INDATE + 90 AS EXPDATE
FROM      PRODUCT;

SELECT    P_CODE, P_INDATE, SYSDATE – 90 AS CUTOFF
FROM      PRODUCT;
```

7-3e Listing Unique Values

- SQL's **DISTINCT** clause produces a list of only those values that are different from one another.
- **DISTINCT** A SQL clause that produces a list of values that are different from one another.

Figure 7.7 A Listing of Distinct V_CODE Values in the PRODUCT Table

V_CODE
21225
21231
21344
23119
24288
25595

```
SELECT      DISTINCT V_CODE
FROM        PRODUCT;
```

7-4 FROM Clause Options

7-4 FROM Clause Options

- The FROM clause of the query specifies the table or tables from which the data is to be retrieved

```
SELECT    P_CODE, P_DESCRIPT, P_INDATE, P_QOH, P_MIN, P_PRICE,
          P_DISCOUNT, V_CODE
FROM      PRODUCT;

SELECT    INV_NUM, P_CODE, LINE_UNITS
FROM      LINE;
```

7-5 ORDER BY Clause Options

7-5 ORDER BY Clause Options

- **ORDER BY** A SQL clause that is useful for ordering the output of a SELECT query (e.g., in ascending or descending order).

```
SELECT      columnlist
FROM        tablelist
[ORDER BY   columnlist [ASC | DESC]];
```

```
SELECT      P_CODE, P_DESCRIPT, P_QOH, P_PRICE
FROM        PRODUCT
ORDER BY    P_PRICE;
```

Figure 7.9 Products Sorted by Price in Ascending Order

P_CODE	P_DESCRIPT	P_QOH	P_PRICE
54778-2T	Rat-tail file, 1/8-in. fine	43	4.99
PVC23DRT	PVC pipe, 3.5-in., 8-ft	188	5.87
SM-18277	1.25-in. metal screw, 25	172	6.99
SW-23116	2.5-in. wd. screw, 50	237	8.45
23109-HB	Claw hammer	23	9.95
23114-AA	Sledge hammer, 12 lb.	8	14.40
13-Q2/P2	7.25-in. pwr. saw blade	32	14.99
14-Q1/L3	9.00-in. pwr. saw blade	18	17.49
2238/QPD	B&D cordless drill, 1/2-in.	12	38.95
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15	39.95
1558-QW1	Hrd. cloth, 1/2-in., 3x50	23	43.99
2232/QWE	B&D jigsaw, 8-in. blade	6	99.87
2232/QTY	B&D jigsaw, 12-in. blade	8	109.92

```
SELECT    P_CODE, P_DESCRIPT, P_QOH, P_PRICE
FROM      PRODUCT
ORDER BY  P_PRICE;
```

Figure 7.10 Telephone List Query Results

EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_AREACODE	EMP_PHONE
Brandon	Marie	G	901	882-0845
Diante	Jorge	D	615	890-4567
Genkazi	Leighla	W	901	569-0093
Johnson	Edward	E	615	898-4387
Jones	Anne	M	615	898-3456
Kolmycz	George	D	615	324-5456
Lange	John	P	901	504-4430
Lewis	Rhonda	G	615	324-4472
Saranda	Hermine	R	615	324-5505
Smith	George	A	615	890-2984
Smith	George	K	901	504-3339
Smith	Jeanine	K	615	324-7883
Smythe	Melanie	P	615	324-9006

```
SELECT      EMP_LNAME, EMP_FNAME, EMP_INITIAL, EMP_AREACODE,
            EMP_PHONE
FROM        EMPLOYEE
ORDER BY    EMP_LNAME, EMP_FNAME, EMP_INITIAL;
```

Figure 7.11 Ordering by a Derived Attribute

P_CODE	P_DESCRIPT	V_CODE	TOTAL
PVC23DRT	PVC pipe, 3.5-in., 8-ft		1103.56
23114-AA	Sledge hammer, 12 lb.		115.20
SM-18277	1.25-in. metal screw, 25	21225	1202.28
23109-HB	Claw hammer	21225	228.85
SW-23116	2.5-in. wd. screw, 50	21231	2002.65
13-Q2/P2	7.25-in. pwr. saw blade	21344	479.68
14-Q1/L3	9.00-in. pwr. saw blade	21344	314.82
54778-2T	Rat-tail file, 1/8-in. fine	21344	214.57
1558-QW1	Hrd. cloth, 1/2-in., 3x50	23119	1011.77
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	23119	599.25
89-WRE-Q	Hicut chain saw, 16 in.	24288	2826.89
2232/QTY	B&D jigsaw, 12-in. blade	24288	879.36
2232/QWE	B&D jigsaw, 8-in. blade	24288	599.22

```
SELECT    P_CODE, P_DESCRIPT, V_CODE, P_PRICE * P_QOH AS TOTAL
FROM      PRODUCT
ORDER BY  V_CODE, TOTAL DESC;
```

Note

If the ordering column has nulls, they are either first or last, depending on the RDBMS. Oracle supports adding a NULLS FIRST or NULLS LAST option to change the sort behavior of nulls in the ORDER BY clause. For example, the following command in Oracle returns the vendor codes sorted from largest to smallest but with the null vendor codes appearing last in the list.

```
SELECT      V_CODE, P_DESCRIPT
FROM        PRODUCT
ORDER BY    V_CODE DESC NULLS LAST;
```

7-6 WHERE Clause Options

7-6a Selecting Rows with Conditional Restrictions

- **WHERE** A SQL clause that adds conditional restrictions to a SELECT statement to limit the rows returned by the query.

```
SELECT      columnlist
FROM        tablelist
[WHERE      conditionlist]
[ORDER BY   columnlist [ASC | DESC]];
```

Figure 7.12 Selected Product Attributes for Vendor Code 21344

P_DESCRIPT	P_QOH	P_PRICE	V_CODE
7.25-in. pwr. saw blade	32	14.99	21344
9.00-in. pwr. saw blade	18	17.49	21344
Rat-tail file, 1/8-in. fine	43	4.99	21344

```
SELECT      P_DESCRIPT, P_QOH, P_PRICE, V_CODE
FROM        PRODUCT
WHERE       V_CODE = 21344;
```

Table 7.5 Comparison Operators

Symbol	Meaning
=	Equal to
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
<> or !=	Not equal to

Figure 7.14 PRODUCT Attributes for VENDOR Codes Other than 21344

P_DESCRIPT	P_QOH	P_PRICE	V_CODE
Power painter, 15 psi., 3-nozzle	8	109.99	25595
Hrd. cloth, 1/4-in., 2x50	15	39.95	23119
Hrd. cloth, 1/2-in., 3x50	23	43.99	23119
B&D jigsaw, 12-in. blade	8	109.92	24288
B&D jigsaw, 8-in. blade	6	99.87	24288
B&D cordless drill, 1/2-in.	12	38.95	25595
Claw hammer	23	9.95	21225
Hicut chain saw, 16 in.	11	256.99	24288
1.25-in. metal screw, 25	172	6.99	21225
2.5-in. wd			
Steel matti			

```
SELECT    P_DESCRIPT, P_QOH, P_PRICE, V_CODE
FROM      PRODUCT
WHERE     V_CODE <> 21344;
```

Figure 7.15 Select PRODUCT Table Attributes with a P_PRICE Restriction

P_DESCRIPT	P_QOH	P_MIN	P_PRICE
Claw hammer	23	10	9.95
Rat-tail file, 1/8-in. fine	43	20	4.99
PVC pipe, 3.5-in., 8-ft	188	75	5.87
1.25-in. metal screw, 25	172	75	6.99
2.5-in. wd. screw, 50	237	100	8.45

```
SELECT    P_DESCRIPT, P_QOH, P_MIN, P_PRICE
FROM      PRODUCT
WHERE     P_PRICE <= 10;
```

7-6b Using Comparison Operators on Character Attributes

- Because computers identify all characters by their numeric American Standard Code for Information Interchange (ASCII) codes, comparison operators may even be used to place restrictions on character-based attributes.

Figure 7.16

```
SELECT    P_CODE, P_DESCRIPT, P_QOH, P_MIN, P_PRICE
FROM      PRODUCT
WHERE     P_CODE < '1558-QW1';
```

P_CODE	P_DESCRIPT	P_QOH	P_MIN	P_PRICE
11QER/31	Power painter, 15 psi., 3-nozzle	8	5	109.99
13-Q2/P2	7.25-in. pwr. saw blade	32	15	14.99
14-Q1/L3	9.00-in. pwr. saw blade	18	12	17.49
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15	8	39.95

7-6c Using Comparison Operators on Dates

- Date procedures are often more software-specific (Access vs Oracle, etc.) than other SQL procedures.

Figure 7.17 Selected PRODUCT Table Attributes: Date Restriction

P_DESCRIPT	P_QOH	P_MIN	P_PRICE	P_INDATE
B&D cordless drill, 1/2-in.	12	5	38.95	20-Jan-22
Claw hammer	23	10	9.95	20-Jan-22
Hicut chain saw, 16 in.	11	5	256.99	07-Feb-22

```
SELECT    P_DESCRIPT, P_QOH, P_MIN, P_PRICE, P_INDATE
FROM      PRODUCT
WHERE     P_INDATE >= '2022-01-20';
```

7-6d Logical Operators: AND, OR, and NOT

- AND, OR, NOT

```
SELECT    P_DESCRIPT, P_QOH, P_PRICE, V_CODE
FROM      PRODUCT
WHERE     V_CODE = 21344 OR V_CODE = 24288;
```

Figure 7.18 The Log

P_DESCRIPT	P_QOH	P_PRICE	V_CODE
7.25-in. pwr. saw blade	32	14.99	21344
9.00-in. pwr. saw blade	18	17.49	21344
B&D jigsaw, 12-in. blade	8	109.92	24288
B&D jigsaw, 8-in. blade	6	99.87	24288
Rat-tail file, 1/8-in. fine	43	4.99	21344
Hicut chain saw, 16 in.	11	256.99	24288

Figure 7.19 The Logical AND

P_DESCRIPT	P_QOH	P_PRICE	V_CODE
Power painter, 15 psi., 3-nozzle	8	109.99	25595
B&D jigsaw, 12-in. blade	8	109.92	24288
Hicut chain saw, 16 in.	11	256.99	24288
Steel matting, 4'x8'x1⁄6", .5" mesh	18	119.95	25595

```
SELECT    P_DESCRIPT, P_QOH, P_PRICE, V_CODE
FROM      PRODUCT
WHERE     P_PRICE > 100
AND       P_QOH < 20;
```

7-6e Special Operators

- ANSI-standard SQL allows the use of special operators with the WHERE clause. These special operators include:
- ***BETWEEN***: Used to check whether an attribute value is within a range
- ***IN***: Used to check whether an attribute value matches any value within a value list
- ***LIKE***: Used to check whether an attribute value matches a given string pattern
- ***IS NULL***: Used to check whether an attribute value is null

The BETWEEN Special Operator

- **BETWEEN** In SQL, a special comparison operator used to check whether a value is within a range of specified values.

```
SELECT      *
FROM        PRODUCT
WHERE       P_PRICE BETWEEN 50.00 AND 100.00;
```

- If your DBMS does not support BETWEEN, you can use:

```
SELECT      *
FROM        PRODUCT
WHERE       P_PRICE => 50.00 AND P_PRICE <= 100.00;
```

Note

When using the BETWEEN special operator, always specify the lower-range value first. The WHERE clause of the command above is interpreted as:

```
WHERE P_PRICE >= 50 AND P_PRICE <= 100
```

If you list the higher-range value first, the DBMS will return an empty result set because the WHERE clause will be interpreted as:

```
WHERE P_PRICE >= 100 AND P_PRICE <= 50
```

Clearly, no product can have a price that is both greater than 100 and simultaneously less than 50. Therefore, no rows can possibly match the criteria.

The IN Special Operator

- **IN** In SQL, a comparison operator used to check whether a value is among a list of specified values

```
SELECT      *
FROM        PRODUCT
WHERE       V_CODE = 21344 OR V_CODE = 24288;
```

can be handled more efficiently with:

```
SELECT      *
FROM        PRODUCT
WHERE       V_CODE IN (21344, 24288);
```

The LIKE Special Operator

- **LIKE** In SQL, a comparison operator used to check whether an attribute's text value matches a specified string pattern.
- Standard SQL allows you to use the percent sign (%) and underscore (_) wildcard characters to make matches when the entire string is not known:

- % means any and all *following* or *preceding* characters are eligible. For example:
  - 'J%' includes Johnson, Jones, Jernigan, July, and J-231Q.
  - 'Jo%' includes Johnson and Jones.
  - '%n' includes Johnson and Jernigan.
- _ means any *one* character may be substituted for the underscore. For example:
  - '_23-456-6789' includes 123-456-6789, 223-456-6789, and 323-456-6789.
  - '_23-_56-678_' includes 123-156-6781, 123-256-6782, and 823-956-6788.
  - '_o_es' includes Jones, Cones, Cokes, totes, and roles.

Figure 7.22 Vendor Contacts That Start with "Smith"

V_NAME	V_CONTACT	V_AREACODE	V_PHONE
Bryson, Inc.	Smithson	615	223-3234
Dome Supply	Smith	901	678-1419
B&K, Inc.	Smith	904	227-0093

```
SELECT    V_NAME, V_CONTACT, V_AREACODE, V_PHONE
FROM      VENDOR
WHERE     V_CONTACT LIKE 'Smith%';
```

The IS NULL Special Operator

- **IS NULL** In SQL, a comparison operator used to check whether an attribute has a value.

Figure 7.23 Products Not Associated with a Vendor

P_CODE	P_DESCRIPT	V_CODE
23114-AA	Sledge hammer, 12 lb.	
PVC23DRT	PVC pipe, 3.5-in., 8-ft	

```
SELECT    P_CODE, P_DESCRIPT, V_CODE
FROM      PRODUCT
WHERE     V_CODE IS NULL;
```

7-7 JOIN Operations

7-7a Natural Join

- A natural join returns all rows with matching values in the matching columns and eliminates duplicate columns.
- SELECT column-list FROM table1 NATURAL JOIN table2
- The natural join performs the following tasks:
  - Determines the common attribute(s) by looking for attributes with identical names and compatible data types.
  - Selects only the rows with common values in the common attribute(s).
  - If there are no common attributes, returns the relational product of the two tables.

Figure 7.25 CUSTOMER NATURAL JOIN INVOICE Results

CUS_CODE	CUS_LNAME	INV_NUMBER	INV_DATE
10011	Dunne	1002	16-Jan-22
10011	Dunne	1004	17-Jan-22
10011	Dunne	1008	17-Jan-22
10012	Smith	1003	16-Jan-22
10014	Orlando	1001	16-Jan-22
10014	Orlando	1006	17-Jan-22
10015	O'Brian	1007	17-Jan-22

```
SELECT    CUS_CODE, CUS_LNAME, INV_NUMBER, INV_DATE
FROM      CUSTOMER NATURAL JOIN INVOICE;
```

Figure 7.26 NATURAL JOIN with Three Tables

INV_NUMBER	P_CODE	P_DESCRIPT	LINE_UNITS	LINE_PRICE
1001	13-Q2/P2	7.25-in. pwr. saw blade	1	14.99
1001	23109-HB	Claw hammer	1	9.95
1002	54778-2T	Rat-tail file, 1/8-in. fine	2	4.99
1003	2238/QPD	B&D cordless drill, 1/2-in.	1	38.95
1003	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	1	39.95
1003	13-Q2/P2	7.25-in. pwr. saw blade	5	14.99
1004	54778-2T	Rat-tail file, 1/8-in. fine	3	4.99
1004	23109-HB	Claw hammer	2	9.95
1005	PVC23DRT	PVC pipe, 3.5-in., 8-ft	12	5.87
1006	SM-18277	1.25-in. metal screw, 25	3	6.99
1006	2232/QTY	B&D jigsaw, 12-in. blade	1	109.92
1006	23109-HB	Claw hammer	1	9.95
1006	89-WRE-Q	Hicut chain saw, 16 in.	1	256.99
1007	13-Q2/P2	7.25-in. pwr. saw blade	2	14.99
1007	54778-2T	Rat-tail file, 1/8-in. fine	1	4.99

```
SELECT    INV_NUMBER, P_CODE, P_DESCRIPT, LINE_UNITS, LINE_PRICE
FROM      INVOICE NATURAL JOIN LINE NATURAL JOIN PRODUCT;
```

7-7b JOIN USING Syntax

- The query returns only the rows with matching values in the column indicated in the USING clause—and that column must exist in both tables
- SELECT *column-list* FROM *table1* JOIN *table2* USING (*common-column*)

Note

Oracle and MySQL support the JOIN USING syntax. MS SQL Server and Access do not. If JOIN USING is used in Oracle, then table qualifiers cannot be used with the common attribute anywhere within the query. MySQL allows table qualifiers on the common attribute anywhere except in the USING clause itself.

Figure 7.27 JOIN USING Results

P_CODE	P_DESCRIPT	V_CODE	V_NAME	V_AREACODE	V_PHONE
23109-HB	Claw hammer	21225	Bryson, Inc.	615	223-3234
SM-18277	1.25-in. metal screw, 25	21225	Bryson, Inc.	615	223-3234
SW-23116	2.5-in. wd. screw, 50	21231	D&E Supply	615	228-3245
13-Q2/P2	7.25-in. pwr. saw blade	21344	Gomez Bros.	615	889-2546
14-Q1/L3	9.00-in. pwr. saw blade	21344	Gomez Bros.	615	889-2546
54778-2T	Rat-tail file, 1/8-in. fine	21344	Gomez Bros.	615	889-2546
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	23119	Randsets Ltd.	901	678-3998
1558-QW1	Hrd. cloth, 1/2-in., 3x50	23119	Randsets Ltd.	901	678-3998
2232/QTY	B&D jigsaw, 12-in. blade	24288	ORDVA, Inc.	615	898-1234
2232/QWE	B&D jigsaw, 8-in. blade	24288	ORDVA, Inc.	615	898-1234
89-WRE-Q	Hicut chain saw, 16 in.	24288	ORDVA, Inc.	615	898-1234
11QER/31	Pow				
2238/QPD	B&D				
WR3/TT3	Ste				

```
SELECT    P_CODE, P_DESCRIPT, V_CODE, V_NAME, V_AREACODE,
          V_PHONE
FROM      PRODUCT JOIN VENDOR USING (V_CODE);
```

7-7c JOIN ON Syntax

- The previous two join styles use common attribute names in the joining tables. Another way to express a join when the tables have no common attribute names is to use the JOIN ON operand.
- SELECT *column-list* FROM *table1* JOIN *table2* ON *join-condition*

```
SELECT    INVOICE.INV_NUMBER, PRODUCT.P_CODE, P_DESCRIPT,
          LINE_UNITS, LINE_PRICE
FROM      INVOICE JOIN LINE ON INVOICE.INV_NUMBER =
          LINE.INV_NUMBER JOIN PRODUCT ON LINE.P_CODE =
          PRODUCT.P_CODE;
```

7-7d Common Attribute Names

- Queries will join tables using PK/FK combinations as the common attribute for the join criteria.
- The NATURAL JOIN and JOIN USING operands automatically eliminate duplicate columns for the common attribute to avoid the issue of duplicate column names.
- The JOIN ON clause does not automatically remove a copy of the common attribute, so it requires a table qualifier whenever the query references the common attribute.

7-7e Old-Style Joins

- Avoid to join using where condition to connect two or more tables.
- See more in book.

7-7f Outer Joins

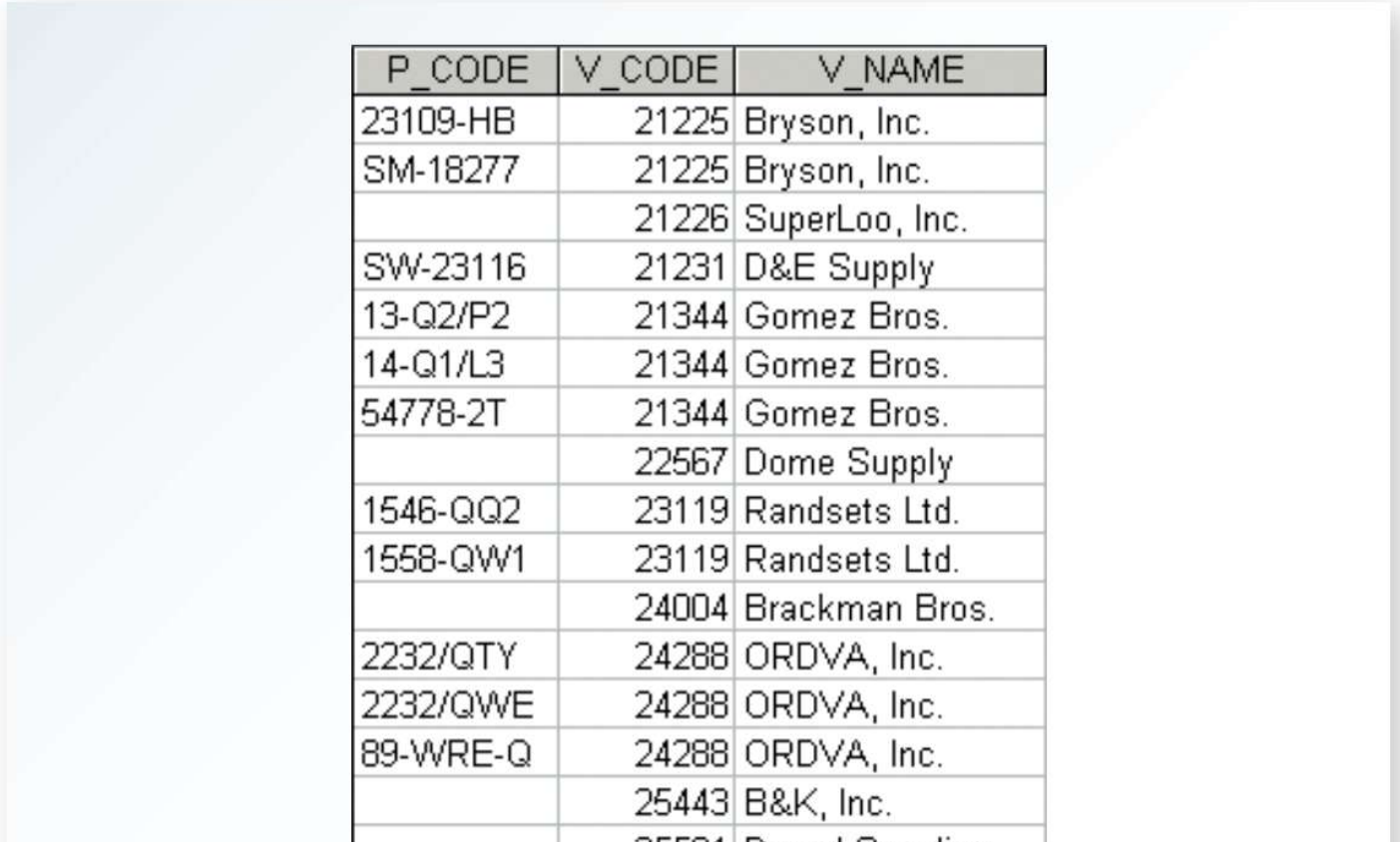
- An outer join returns not only the rows matching the join condition (that is, rows with matching values in the common columns), but also the rows with unmatched values.
- The left outer join returns not only the rows matching the join condition (that is, rows with matching values in the common column), but also the rows in the left table with unmatched values in the right table.
- The right outer join returns not only the rows matching the join condition (that is, rows with matching values in the common column), but also the rows in the right table with unmatched values in the left table
- The full outer join returns not only the rows matching the join condition (that is, rows with matching values in the common column), but also all of the rows with unmatched values in the table on either side.

```
SELECT    column-list
FROM      table1 LEFT [OUTER] JOIN table2 ON join-condition

SELECT    column-list
FROM      table1 RIGHT [OUTER] JOIN table2 ON join-condition

SELECT    column-list
FROM      table1 FULL [OUTER] JOIN table2 ON join-condition
```

Figure 7.30 LEFT JOIN Results



P_CODE	V_CODE	V_NAME
23109-HB	21225	Bryson, Inc.
SM-18277	21225	Bryson, Inc.
	21226	SuperLoo, Inc.
SW-23116	21231	D&E Supply
13-Q2/P2	21344	Gomez Bros.
14-Q1/L3	21344	Gomez Bros.
54778-2T	21344	Gomez Bros.
	22567	Dome Supply
1546-QQ2	23119	Randsets Ltd.
1558-QW1	23119	Randsets Ltd.
	24004	Brackman Bros.
2232/QTY	24288	ORDVA, Inc.
2232/QWE	24288	ORDVA, Inc.
89-WRE-Q	24288	ORDVA, Inc.
	25443	B&K, Inc.

```
SELECT    P_CODE, VENDOR.V_CODE, V_NAME
FROM      VENDOR LEFT JOIN PRODUCT ON VENDOR. V_CODE =
          PRODUCT.V_CODE;
```

Figure 7.31 RIGHT JOIN Results

P_CODE	V_CODE	V_NAME
23114-AA		
PVC23DRT		
23109-HB	21225	Bryson, Inc.
SM-18277	21225	Bryson, Inc.
SW-23116	21231	D&E Supply
13-Q2/P2	21344	Gomez Bros.
14-Q1/L3	21344	Gomez Bros.
54778-2T	21344	Gomez Bros.
1546-QQ2	23119	Randsets Ltd.
1558-QW1	23119	Randsets Ltd.
2232/QTY	24288	ORDVA, Inc.
2232/QWE	24288	ORDVA, Inc.
89-WRE-Q	24288	ORDVA, Inc.

```
SELECT    P_CODE, VENDOR.V_CODE, V_NAME
FROM      VENDOR RIGHT JOIN PRODUCT ON VENDOR. V_CODE =
          PRODUCT.V_CODE;
```

Figure 7.32 Vendors Not Associated with Any Product

V_CODE	V_NAME	P_CODE
21226	SuperLoo, Inc.	
22567	Dome Supply	
24004	Brackman Bros.	
25443	B&K, Inc.	

```
SELECT    V_CODE, V_NAME, P_CODE
FROM      PRODUCT RIGHT JOIN VENDOR ON PRODUCT.V_CODE =
          VENDOR.V_CODE
WHERE     P_CODE IS NULL;
```

Figure 7.33 FULL OUTER JOIN Results

P_CODE	V_CODE	V_NAME
	21226	SuperLoo, Inc.
	22567	Dome Supply
	24004	Brackman Bros.
	25443	B&K, Inc.
	25501	Damal Supplies
11QER/31	25595	Rubicon Systems
13-Q2/P2	21344	Gomez Bros.
14-Q1/L3	21344	Gomez Bros.

```
SELECT    P_CODE, VENDOR.V_CODE, V_NAME
FROM      VENDOR FULL JOIN PRODUCT ON VENDOR. V_CODE =
          PRODUCT.V_CODE;
```

2238/QPD	25595	Rubicon Systems
23109-HB	21225	Bryson, Inc.
23114-AA		
54778-2T	21344	Gomez Bros.
89-WRE-Q	24288	ORDVA, Inc.
PVC23DRT		
SM-18277	21225	Bryson, Inc.
SW-23116	21231	D&E Supply
WR3/TT3	25595	Rubicon Systems

7-7g Cross Join

- A **cross join** performs a relational product (also known as the *Cartesian product*) of two tables.
- SELECT *column-list* FROM *table1* CROSS JOIN *table2*
  - SELECT * FROM INVOICE CROSS JOIN LINE;

You can also perform a cross join that yields only specified attributes. For example, you can specify:

```
SELECT    INVOICE.INV_NUMBER, CUS_CODE, INV_DATE, P_CODE
FROM      INVOICE CROSS JOIN LINE;
```

The results generated through that SQL statement can also be generated by using the following syntax:

```
SELECT    INVOICE.INV_NUMBER, CUS_CODE, INV_DATE, P_CODE
FROM      INVOICE, LINE;
```

7-7h Joining Tables with an Alias

```
SELECT    P_DESCRIPT, P_PRICE, V_NAME, V_CONTACT, V_AREACODE,
          V_PHONE
FROM      PRODUCT P JOIN VENDOR V ON P.V_CODE = V.V_CODE;
```

Note

MS Access requires the AS keyword before a table alias. Oracle and MySQL do not use the AS keyword for a table alias, while MS SQL Server will accept table aliases with or without the AS keyword. Using the AS keyword would change the above query to:

```
SELECT    P_DESCRIPT, P_PRICE, V_NAME, V_CONTACT, V_AREACODE, V_PHONE
FROM      PRODUCT AS P JOIN VENDOR AS V ON P.V_CODE = V.V_CODE;
```

7-7i Recursive Joins

- **recursive query** A query that joins a table to itself.
- Using the data in the EMP table, you can generate a list of all employees with their managers' names by joining the EMP table to itself.

```
SELECT    E.EMP_NUM, E.EMP_LNAME, E.EMP_MGR, M.EMP_LNAME
FROM      EMP E JOIN EMP M ON E.EMP_MGR = M.EMP_NUM;
```

Figure 7.35 Using an Alias to Join a Table to Itself

EMP_NUM	E.EMP_LNAME	EMP_MGR	M.EMP_LNAME
112	Johnson	100	Kolmycz
103	Jones	100	Kolmycz
102	Vandam	100	Kolmycz
101	Lewis	100	Kolmycz
115	Saranda	105	Williams
113	Smythe	105	Williams
111	Washington	105	Williams
107	Diante	105	Williams
106	Smith	105	Williams
104	Lange	105	Williams
116	Smith	108	Wiesenbach
114	Brandon	108	Wiesenbach
110	Genkazi	108	Wiesenbach

```
SELECT    E.EMP_NUM, E.EMP_LNAME, E.EMP_MGR, M.EMP_LNAME
FROM      EMP E JOIN EMP M ON E.EMP_MGR = M.EMP_NUM;
```

7-8 Aggregate Processing

7-8a Aggregate Functions

Table 7.7 Some Basic SQL Aggregate Functions

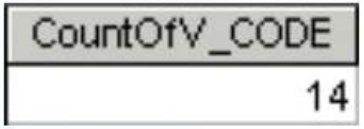
Function	Output
COUNT	The number of rows containing non-null values
MIN	The minimum attribute value encountered in a given column
MAX	The maximum attribute value encountered in a given column
SUM	The sum of all values for a given column
AVG	The arithmetic mean (average) for a specified column

Figure 7.37 Count of Product Codes in the PRODUCT Table

CountOfP_CODE
16

```
SELECT      COUNT(P_CODE)
FROM        PRODUCT;
```

Figure 7.38 Count of Vendor Codes in the PRODUCT Table



CountOfV_CODE
14

```
SELECT      COUNT(V_CODE)
FROM        PRODUCT;
```

```
SELECT    COUNT(DISTINCT V_CODE) AS "COUNT DISTINCT"
FROM      PRODUCT;
```

Figure 7.39 Count of Distinct Vendor Codes

Count Distinct
6

Note

MS Access does not support the use of COUNT with the DISTINCT clause. If you want to use such queries in MS Access, you must create subqueries (discussed later in this chapter) with DISTINCT and NOT NULL clauses. For example, the equivalent MS Access queries for the two queries above are:

```
SELECT      COUNT(*)
FROM        (SELECT DISTINCT V_CODE FROM PRODUCT WHERE V_CODE IS NOT NULL);
```

and

```
SELECT      COUNT(*)
FROM        (SELECT DISTINCT V_CODE
            FROM (SELECT V_CODE, P_PRICE FROM PRODUCT
                  WHERE V_CODE IS NOT NULL AND P_PRICE < 10));
```

Subqueries are discussed in detail later in this chapter.

MIN and MAX

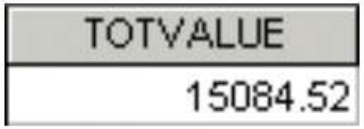
Figure 7.40 Maximum and Minimum Price Output

MAXPRICE	MINPRICE
256.99	4.99

```
SELECT    MAX(P_PRICE) AS MAXPRICE, MIN(P_PRICE) AS MINPRICE
FROM      PRODUCT;
```

SUM and AVG

Figure 7.41 Total Value of All Items in the PRODUCT Table



TOTVALUE
15084.52

```
SELECT    SUM(P_QOH * P_PRICE) AS TOTVALUE
FROM      PRODUCT;
```

Figure 7.42 Average Product Price

AVGPRICE
56.42125

```
SELECT    AVG(P_PRICE) AS AVGPRICE
FROM      PRODUCT;
```

7-8b Grouping Data

- Rows can be grouped into smaller collections quickly and easily using the **GROUP BY** clause within the SELECT statement.
- The aggregate functions will then summarize the data within each smaller collection.

```
SELECT      columnlist
FROM        tablelist
[WHERE      conditionlist]
[GROUP BY   columnlist]
[ORDER BY   columnlist [ASC | DESC]];
```

```
SELECT      V_CODE, AVG(P_PRICE) AS AVGPRICE
FROM        PRODUCT
GROUP BY    V_CODE;
```

Figure 7.43 Average Price of Products from Each Vendor

V_CODE	AVGPRICE
	10.13
21225	8.47
21231	8.45
21344	12.49
23119	41.97
24288	155.59
25595	89.63

7-8c HAVING Clause

- **HAVING** A clause applied to the output of a GROUP BY operation to restrict selected rows.

```
SELECT      columnlist
FROM        tablelist
[WHERE      conditionlist]
[GROUP BY   columnlist]
[HAVING     conditionlist]
[ORDER BY   columnlist [ASC | DESC]];
```

```
SELECT      V_CODE, COUNT(P_CODE) AS NUMPRODS
FROM        PRODUCT
GROUP BY    V_CODE
HAVING      AVG(P_PRICE) < 10
ORDER BY    V_CODE;
```

Figure 7.48 Application of the HAVING Clause

V_CODE	NUMPRODS
21225	2
21231	1

7-9 Subqueries

7-9 Subqueries

- **Subquery** A query that is embedded (or nested) inside another query. Also known as a *nested query* or an *inner query*.

```
SELECT      V_CODE, V_NAME
FROM        VENDOR
WHERE       V_CODE NOT IN (SELECT V_CODE FROM PRODUCT WHERE
            V_CODE IS NOT NULL);

SELECT      P_CODE, P_PRICE
FROM        PRODUCT
WHERE       P_PRICE >= (SELECT AVG(P_PRICE) FROM PRODUCT);
```

- A subquery is a query (SELECT statement) inside another query.
- A subquery is normally expressed inside parentheses.
- The first query in the SQL statement is known as the *outer query*.
- The query inside the SQL statement is known as the *inner query*.
- The inner query is executed first.
- The output of an inner query is used as the input for the outer query.
- The entire SQL statement is sometimes referred to as a *nested query*.

7-9a WHERE Subqueries

```
SELECT    P_CODE, P_PRICE
FROM      PRODUCT
WHERE     P_PRICE >= (SELECT AVG(P_PRICE) FROM PRODUCT);

SELECT    DISTINCT CUS_CODE, CUS_LNAME, CUS_FNAME
FROM      CUSTOMER JOIN INVOICE USING (CUS_CODE)
          JOIN LINE USING (INV_NUMBER)
          JOIN PRODUCT USING (P_CODE)
WHERE     P_CODE = (SELECT P_CODE FROM PRODUCT WHERE
          P_ DESCRIPT = 'Claw hammer');
```

7-9b IN Subqueries

```
SELECT    DISTINCT CUSTOMER.CUS_CODE, CUS_LNAME, CUS_FNAME
FROM      CUSTOMER JOIN INVOICE ON CUSTOMER.CUS_CODE =
          INVOICE.CUS_CODE
          JOIN LINE ON INVOICE.INV_NUMBER = LINE.INV_NUMBER
          JOIN PRODUCT ON LINE.P_CODE = PRODUCT.P_CODE
WHERE     P_CODE IN    (SELECT P_CODE FROM PRODUCT
                       WHERE P_DESCRIPT LIKE '%hammer%'
                       OR P_DESCRIPT LIKE '%saw%');
```

7-9c HAVING Subqueries

```
SELECT      P_CODE, SUM(LINE_UNITS) AS TOTALUNITS
FROM        LINE
GROUP BY    P_CODE
HAVING      SUM(LINE_UNITS) > (SELECT AVG(LINE_UNITS) FROM LINE);
```

7-9d Multirow Subquery Operators: ALL and ANY

- which products cost more than all individual products provided by vendors from Florida

```
SELECT    P_CODE, P_QOH * P_PRICE AS TOTALVALUE
FROM      PRODUCT
WHERE     P_QOH * P_PRICE > ALL (SELECT P_QOH * P_PRICE
                                 FROM PRODUCT
                                 WHERE V_CODE IN (SELECT V_CODE
                                                  FROM VENDOR WHERE
                                                  V_STATE = 'FL'));
```

7-9e FROM Subqueries

```
SELECT    DISTINCT CUSTOMER.CUS_CODE, CUSTOMER.CUS_LNAME
FROM      CUSTOMER JOIN
              (SELECT INVOICE.CUS_CODE FROM INVOICE JOIN LINE
              ON INVOICE.INV_NUMBER = LINE.INV_NUMBER WHERE
              P_CODE = '13-Q2/P2') CP1
          ON CUSTOMER.CUST_CODE = CP1.CUS_CODE
          JOIN
              (SELECT INVOICE.CUS_CODE FROM INVOICE JOIN LINE
              ON INVOICE.INV_NUMBER = LINE.INV_NUMBER WHERE
              P_CODE = '23109-HB') CP2
          ON CP1.CUS_CODE = CP2.CUS_CODE;
```

7-10 SQL Functions

Review Questions

1. Explain why it would be preferable to use a DATE data type to store date data instead of a character data type.
2. Explain why the following command would create an error and what changes could be made to fix the error:

   SELECT V_CODE, SUM(P_QOH) FROM PRODUCT;
3. What is a cross join? Give an example of its syntax.
4. What three join types are included in the outer join classification?
5. Using tables named T1 and T2, write a query example for each of the three join types you described in Question 4. Assume that T1 and T2 share a common column named C1.
6. What is a recursive join?
7. Rewrite the following WHERE clause without the use of the IN special operator:

   WHERE V_STATE IN ('TN', 'FL', 'GA')
8. Explain the difference between an ORDER BY clause and a GROUP BY clause.
9. Explain why the following two commands produce different results:

   SELECT DISTINCT COUNT (V_CODE) FROM PRODUCT;

   SELECT COUNT (DISTINCT V_CODE) FROM PRODUCT;
10. What is the difference between the COUNT aggregate function and the SUM aggregate function?
11. In a SELECT query, what is the difference between a WHERE clause and a HAVING clause?
12. What is a subquery, and what are its basic characteristics?
13. What are the three types of results that a subquery can return?

14. What is a correlated subquery? Give an example.
15. Explain the difference between a regular subquery and a correlated subquery.
16. What does it mean to say that SQL operators are set-oriented?
17. The relational set operators UNION, INTERSECT, and EXCEPT (MINUS) work properly only when the relations are union-compatible. What does union-compatible mean, and how would you check for this condition?
18. What is the difference between UNION and UNION ALL? Write the syntax for each.
19. Suppose you have two tables: EMPLOYEE and EMPLOYEE_1. The EMPLOYEE table contains the records for three employees: Alice Cordoza, John Cretchakov, and Anne McDonald. The EMPLOYEE_1 table contains the records for employees John Cretchakov and Mary Chen. Given that information, list the query output for the UNION query.
20. Given the employee information in Question 19, list the query output for the UNION ALL query.
21. Given the employee information in Question 19, list the query output for the INTERSECT query.
22. Given the employee information in Question 19, list the query output for the EXCEPT (MINUS) query of EMPLOYEE to EMPLOYEE_1.

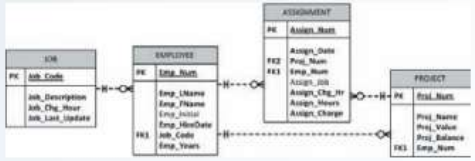
23. Suppose a PRODUCT table contains two attributes, PROD_CODE and VEND_CODE. Those two attributes have values of ABC, 125, DEF, 124, GHI, 124, and JKL, 123, respectively. The VENDOR table contains a single attribute, VEND_CODE, with values 123, 124, 125, and 126, respectively. (The VEND_CODE attribute in the PRODUCT table is a foreign key to the VEND_CODE in the VENDOR table.) Given that information, what would be the query output for:
    a. A UNION query based on the two tables?
    b. A UNION ALL query based on the two tables?
    c. An INTERSECT query based on the two tables?
    d. An EXCEPT (MINUS) query based on the two tables?
24. Why does the order of the operands (tables) matter in an EXCEPT (MINUS) query but not in a UNION query?
25. What MS Access and SQL Server function should you use to calculate the number of days between your birth date and the current date?
26. What Oracle function should you use to calculate the number of days between your birth date and the current date?
27. What string function should you use to list the first three characters of a company's EMP_LNAME values? Give an example using a table named EMPLOYEE. Provide examples for Oracle and SQL Server.
28. What two things must a SQL programmer understand before beginning to craft a SELECT query?

Problems

The Ch07_ConstructCo database stores data for a consulting company that tracks all charges to projects. The charges are based on the hours each employee works on each project. The structure and contents of the Ch07_ConstructCo database are shown in Figure P7.1.

Figure P7.1 The Ch07_ConstructCo Database

Relational diagram



Database name: Ch07_ConstructCo

Table name: JOB

JOB_CODE	JOB_DESCRIPTION	JOB_CHG_HOUR	JOB_LAST_UPDATE
500	Programmer	35.75	20-Nov-21
501	Systems Analyst	96.75	20-Nov-21
502	Database Designer	125.00	24-Mar-22
503	Electrical Engineer	84.50	20-Nov-21
504	Mechanical Engineer	67.90	20-Nov-21
505	Civil Engineer	55.78	20-Nov-21
506	Clerical Support	26.87	20-Nov-21
507	DSS Analyst	45.95	20-Nov-21
508	Applications Designer	48.10	24-Mar-22
509	Bio Technician	34.55	20-Nov-21
510	General Support	18.36	20-Nov-21

Table name: PROJECT

PROJ_NUM	PROJ_NAME	PROJ_VALUE	PROJ_BALANCE	EMP_NUM
15	Evergreen	1453500.00	1002350.00	103
18	Amber Wave	3500500.00	2110348.00	108
22	Rolling Tide	805000.00	500345.20	102
25	Starflight	2650500.00	2309880.00	107

Table name: EMPLOYEE

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE	EMP_YEARS
101	News	John	G	08-Nov-04	502	17
102	Senior	David	H	12-Jul-83	501	28
103	Arbough	June	E	01-Dec-00	500	21
104	Ramoras	Anne	K	15-Nov-91	501	30
105	Johnson	Alice	K	01-Feb-97	502	25
106	Smithfield	William		22-Jun-08	500	13
107	Alonzo	Maria	D	10-Oct-97	500	24
108	Washington	Ralph	B	22-Aug-95	501	26
109	Smith	Larry	W	18-Jul-01	501	20
110	Olenko	Gerald	A	11-Dec-99	505	22
111	Wabash	Geoff	B	04-Apr-95	506	27
112	Smithson	Darlene	M	23-Oct-98	507	23
113	Joenbrood	Delbert	K	15-Nov-00	508	21
114	Jones	Annelise		20-Aug-97	508	24
115	Bawangi	Travis	B	25-Jan-96	501	26
116	Pratt	Gerald	L	05-Mar-01	510	21
117	Williamson	Angie	H	19-Jun-00	509	21
118	Frommer	James	J	04-Jan-09	510	13

Table name: ASSIGNMENT

ASSIGN_NUM	ASSIGN_DATE	PROJ_NUM	EMP_NUM	ASSIGN_JOB	ASSIGN_CHG_HR	ASSIGN_HOURS	ASSIGN_CHARGE
1001	22-Mar-22	18	103	503	84.50	3.5	295.75
1002	22-Mar-22	22	117	509	34.55	4.2	145.11
1003	22-Mar-22	18	117	509	34.55	2.0	69.10
1004	22-Mar-22	18	103	503	84.50	5.9	498.55
1005	22-Mar-22	25	108	501	96.75	2.2	212.85
1006	22-Mar-22	22	104	501	96.75	4.2	406.35
1007	22-Mar-22	25	113	508	58.75	3.8	192.95
1008	22-Mar-22	18	103	503	84.50	0.9	76.05
1009	23-Mar-22	15	115	501	96.75	5.6	541.80
1010	23-Mar-22	15	117	509	34.55	2.4	82.92
1011	23-Mar-22	25	105	502	105.00	4.3	451.50
1012	23-Mar-22	18	108	501	96.75	3.4	328.95
1013	23-Mar-22	25	115	501	96.75	2.0	193.50
1014	23-Mar-22	22	104	501	96.75	2.8	270.90
1015	23-Mar-22	15	103	503	84.50	6.1	515.45
1016	23-Mar-22	22	105	502	105.00	4.7	493.50
1017	23-Mar-22	18	117	509	34.55	3.8	131.29
1018	23-Mar-22	25	117	509	34.55	2.2	76.01
1019	24-Mar-22	25	104	501	110.50	4.9	541.45
1020	24-Mar-22	15	101	502	125.00	3.1	387.50
1021	24-Mar-22	22	108	501	110.50	2.7	298.35
1022	24-Mar-22	22	115	501	110.50	4.9	541.45
1023	24-Mar-22	22	105	502	125.00	3.5	437.50
1024	24-Mar-22	15	103	503	84.50	3.3	278.85
1025	24-Mar-22	18	117	509	34.55	4.2	145.11

Note that the ASSIGNMENT table in Figure P7.1 stores the JOB_CHG_HOUR values as an attribute (ASSIGN_CHG_HR) to maintain historical accuracy of the data. The JOB_CHG_HOUR values are likely to change over time. In fact, a JOB_CHG_HOUR change will be reflected in the ASSIGNMENT table. Naturally, the employee primary job assignment might also change, so the ASSIGN_JOB is also stored. Because those attributes are required to maintain the historical accuracy of the data, they are *not* redundant.

Given the structure and contents of the Ch07_ConstructCo database shown in Figure P7.1, use SQL commands to answer the following problems.

1. Write the SQL code required to list the employee number, last name, first name, and middle initial of all employees whose last names start with *Smith*. In other words, the rows for both Smith and Smithfield should be included in the listing. Sort the results by employee number. Assume case sensitivity.
2. Using the EMPLOYEE, JOB, and PROJECT tables in the Ch07_ConstructCo database, write the SQL code that will join the EMPLOYEE and PROJECT tables using EMP_NUM as the common attribute. Display the attributes shown in the results presented in Figure P7.2, sorted by project value.

Figure P7.2 The Query Results for Problem 2

PROJ_NAME	PROJ_VALUE	PROJ_BALANCE	EMP_LNAME	EMP_FNAME	EMP_INITIAL	JOB_CODE	JOB_DESCRIPTION	JOB_CHG_HOUR
Rolling Tide	805000.00	500345.20	Senior	David	H	501	Systems Analyst	96.75
Evergreen	1453500.00	1002350.00	Arbough	June	E	500	Programmer	35.75
Starflight	2650500.00	2309880.00	Alonzo	Maria	D	500	Programmer	35.75
Amber Wave	3500500.00	2110346.00	Washington	Ralph	B	501	Systems Analyst	96.75

3. Write the SQL code that will produce the same information that was shown in Problem 2 but sorted by the employee's last name.
4. Write the SQL code that will list only the distinct project numbers in the ASSIGNMENT table, sorted by project number.
5. Write the SQL code to validate the ASSIGN_CHARGE values in the ASSIGNMENT table. Your query should retrieve the assignment number, employee number, project number, the stored assignment charge (ASSIGN_CHARGE), and the calculated assignment charge (calculated by multiplying ASSIGN_CHG_HR by ASSIGN_HOURS). Sort the results by the assignment number.
6. Using the data in the ASSIGNMENT table, write the SQL code that will yield the total number of hours worked for each employee and the total charges stemming from those hours worked, sorted by employee number. The results of running that query are shown in Figure P7.6.

Figure P7.6 Total Hours and Charges by Employee

EMP_NUM	EMP_LNAME	SumOfASSIGN_HOURS	SumOfASSIGN_CHARGE
101	News	3.1	387.50
103	Arbough	19.7	1664.65
104	Ramoras	11.9	1218.70
105	Johnson	12.5	1382.50
108	Washington	8.3	840.15
113	Joenbrood	3.8	192.85
115	Bawangi	12.5	1276.75
117	Williamson	18.8	649.54

7. Write a query to produce the total number of hours and charges for each of the projects represented in the ASSIGNMENT table, sorted by project number. The output is shown in Figure P7.7.

Figure P7.7 Total Hours and Charges by Project

PROJ_NUM	SumOfASSIGN_HOURS	SumOfASSIGN_CHARGE
15	20.5	1806.52
18	23.7	1544.80
22	27.0	2593.16
25	19.4	1668.16

8. Write the SQL code to generate the total hours worked and the total charges made by all employees. The results are shown in Figure P7.8.

Figure P7.8 Total Hours and Charges, All Employees

SumOfSumOfASSIGN_HOURS	SumOfSumOfASSIGN_CHARGE
90.6	7612.64

Figure P7.9 The Ch07_SaleCo Database

Relational diagram

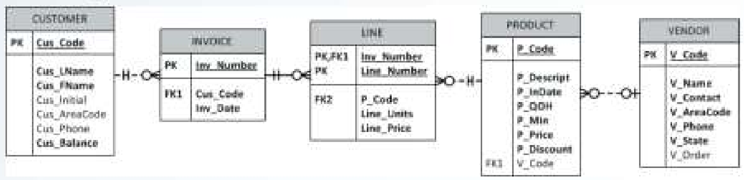


Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_BALANCE
10010	Ramas	Alfred	A	615	844-2573	0.00
10011	Dunne	Leona	K	713	894-1238	0.00
10012	Smith	Kathy	W	615	894-2285	345.86
10013	Olowski	Paul	F	615	894-2180	536.75
10014	Orlando	Myron		615	222-1672	0.00
10015	O'Brian	Amy	B	713	442-3381	0.00
10016	Brown	James	G	615	297-1228	221.19
10017	Williams	George		615	290-2556	768.93
10018	Farriss	Anne	G	713	382-7185	216.55
10019	Smith	Olette	K	615	297-3809	0.00

Table name: VENDOR

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
21225	Bryson, Inc.	Smithson	615	223-3234	TN	Y
21226	SuperLoo, Inc.	Flushing	904	215-8995	FL	N
21231	D&E Supply	Singh	615	228-3245	TN	Y
21344	Gomez Bros.	Ortega	615	889-2546	KY	N
22567	Dome Supply	Smith	901	678-1419	GA	N
23119	Randsets Ltd.	Anderson	901	678-3998	GA	Y
24004	Brackman Bros.	Browning	615	228-1410	TN	N
24288	ORDVA, Inc.	Hakford	615	898-1234	TN	Y
25443	B&K, Inc.	Smith	904	227-0093	FL	N
25501	Damal Supplies	Smythe	615	890-3529	TN	N
25595	Rubicon Systems	Orton	904	456-0092	FL	Y

Table name: INVOICE

INV_NUMBER	CUS_CODE	INV_DATE
1001	10014	16-Jan-22
1002	10011	16-Jan-22
1003	10012	16-Jan-22
1004	10011	17-Jan-22
1005	10018	17-Jan-22
1006	10014	17-Jan-22
1007	10015	17-Jan-22
1008	10011	17-Jan-22

Table name: LINE

INV_NUMBER	LINE_NUMBER	P_CODE	LINE_UNITS	LINE_PRICE
1001	1	13-Q2/P2	1	14.99
1001	2	23109-HB	1	9.95
1002	1	54778-2T	2	4.99
1003	1	2238/QPD	1	38.95
1003	2	1546-QQ2	1	39.95
1003	3	13-Q2/P2	5	14.99
1004	1	54778-2T	3	4.99
1004	2	23109-HB	2	9.95
1005	1	PVC23DRT	12	5.87
1006	1	SM-18277	3	6.99
1006	2	2232/QTY	1	109.92
1006	3	23109-HB	1	9.95
1006	4	89-WRE-Q	1	256.99
1007	1	13-Q2/P2	2	14.99
1007	2	54778-2T	1	4.99
1008	1	PVC23DRT	5	5.87
1008	2	WR3/TT3	3	119.95
1008	3	23109-HB	1	9.95

Table name: PRODUCT

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-21	8	5	109.99	0.00	25595
13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-21	32	15	14.99	0.05	21344
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-21	18	12	17.49	0.00	21344
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-22	15	8	39.95	0.00	23119
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-22	23	5	43.99	0.00	23119
2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-21	8	5	109.92	0.05	24288
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-21	6	5	99.87	0.05	24288
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-22	12	5	38.95	0.05	25595
23109-HB	Claw hammer	20-Jan-22	23	10	9.95	0.10	21225
23114-AA	Sledge hammer, 12 lb.	02-Jan-22	8	5	14.40	0.05	
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-21	43	20	4.99	0.00	21344
89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-22	11	5	256.99	0.05	24288
PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-22	188	75	5.87	0.00	
SM-18277	1.25-in. metal screw, 25	01-Mar-22	172	75	6.99	0.00	21225
SW-23116	2.5-in. wd. screw, 50	24-Feb-22	237	100	8.45	0.00	21231
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-22	18	5	119.95	0.10	25595

9. Write a query to count the number of invoices.
10. Write a query to count the number of customers with a balance of more than $500.
11. Generate a listing of all purchases made by the customers, using the output shown in Figure P7.11 as your guide. Sort the results by customer code, invoice number, and product description.

Figure P7.11 List of Customer Purchases

CUS_CODE	INV_NUMBER	INV_DATE	P_DESCRIPT	LINE_UNITS	LINE_PRICE
10011	1002	16-Jan-22	Rat-tail file, 1/8-in. fine	2	4.99
10011	1004	17-Jan-22	Claw hammer	2	9.95
10011	1004	17-Jan-22	Rat-tail file, 1/8-in. fine	3	4.99
10011	1008	17-Jan-22	Claw hammer	1	9.95
10011	1008	17-Jan-22	PVC pipe, 3.5-in., 8-ft	5	5.87
10011	1008	17-Jan-22	Steel matting, 4'x8'x1/6", .5" mesh	3	119.95
10012	1003	16-Jan-22	7.25-in. pwr. saw blade	5	14.99
10012	1003	16-Jan-22	B&D cordless drill, 1/2-in.	1	38.95
10012	1003	16-Jan-22	Hrd. cloth, 1/4-in., 2x50	1	39.95
10014	1001	16-Jan-22	7.25-in. pwr. saw blade	1	14.99
10014	1001	16-Jan-22	Claw hammer	1	9.95
10014	1006	17-Jan-22	1.25-in. metal screw, 25	3	6.99
10014	1006	17-Jan-22	B&D jigsaw, 12-in. blade	1	109.92
10014	1006	17-Jan-22	Claw hammer	1	9.95
10014	1006	17-Jan-22	Hicut chain saw, 16 in.	1	256.99
10015	1007	17-Jan-22	7.25-in. pwr. saw blade	2	14.99
10015	1007	17-Jan-22	Rat-tail file, 1/8-in. fine	1	4.99
10018	1005	17-Jan-22	PVC pipe, 3.5-in., 8-ft	12	5.87

12. Using the output shown in Figure P7.12 as your guide, generate a list of customer purchases, including the subtotals for each of the invoice line numbers. The subtotal is a derived attribute calculated by multiplying LINE_UNITS by LINE_PRICE. Sort the output by customer code, invoice number, and product description. Be certain to use the column aliases as shown in the figure.

Figure P7.13 Customer Purchase Summary

CUS_CODE	CUS_BALANCE	Total Purchases
10011	0.00	444.00
10012	345.86	153.86
10014	0.00	422.77
10015	0.00	34.97
10018	216.55	70.44

14. Modify the query in Problem 13 to include the number of individual product purchases made by each customer. (In other words, if the customer's invoice is based on three products, one per LINE_NUMBER, you count three product purchases. Note that in the original invoice data, customer 10011 generated three invoices, which contained a total of six lines, each representing a product purchase.) Your output values must match those shown in Figure P7.14, sorted by customer code.

Figure P7.14 Customer Total Purchase Amounts and Number of Purchases

CUS_CODE	CUS_BALANCE	Total Purchases	Number of Purchases
10011	0.00	444.00	6
10012	345.86	153.86	3
10014	0.00	422.77	6
10015	0.00	34.97	2
10018	216.55	70.44	1

15. Use a query to compute the total of all purchases, the number of purchases, and the average purchase amount made by each customer. Your output values must match those shown in Figure P7.15. Sort the results by customer code.

Figure P7.15 Average Purchase Amount by Customer

CUS_CODE	CUS_BALANCE	Total Purchases	Number of Purchases	Average Purchase Amount
10011	0.00	444.00	6	74.00
10012	345.86	153.86	3	51.28
10014	0.00	422.77	6	70.46
10015	0.00	34.97	2	17.48
10018	216.55	70.44	1	70.44

16. Create a query to produce the total purchase per invoice, generating the results shown in Figure P7.16, sorted by invoice number. The invoice total is the sum of the product purchases in the LINE that corresponds to the INVOICE.

Figure P7.16 Invoice Totals

INV_NUMBER	Invoice Total
1001	24.94
1002	9.98
1003	153.85
1004	34.87
1005	70.44
1006	397.83
1007	34.97
1008	399.15

17. Use a query to show the invoices and invoice totals in Figure P7.17. Sort the results by customer code and then by invoice number.

Figure P7.17 Invoice Totals by Customer

CUS_CODE	INV_NUMBER	Invoice Total
10011	1002	9.98
10011	1004	34.87
10011	1008	399.15
10012	1003	153.85
10014	1001	24.94
10014	1006	397.83
10015	1007	34.97
10018	1005	70.44

18. Write a query to produce the number of invoices and the total purchase amounts by customer, using the output shown in Figure P7.18 as your guide. Note the results are sorted by customer code. (Compare this summary to the results shown in Problem 17.)

Figure P7.18 Number of Invoices and Total Purchase Amounts by Customer

CUS_CODE	Number of Invoices	Total Customer Purchases
10011	3	444.00
10012	1	153.85
10014	2	422.77
10015	1	34.97
10018	1	70.44

19. Write a query to generate the total number of invoices, the invoice total for all of the invoices, the smallest of the customer purchase amounts, the largest of the customer purchase amounts, and the average of all the customer purchase amounts. Your output must match Figure P7.19.

Figure P7.19 Number of Invoices, Invoice Totals, Minimum, Maximum, and Average Sales

Total Invoices	Total Sales	Minimum Customer Purchases	Largest Customer Purchases	Average Customer Purchases
8	1126.03	34.97	444.00	225.21

20. List the balances of customers who have made purchases during the current invoice cycle—that is, for the customers who appear in the INVOICE table. The results of this query are shown in Figure P7.20, sorted by customer code.

Figure P7.20 Balances for Customers Who Made Purchases

CUS_CODE	CUS_BALANCE
10011	0.00
10012	345.86
10014	0.00
10015	0.00
10018	216.55

21. Provide a summary of customer balance characteristics for customers who made purchases. Include the minimum balance, maximum balance, and average balance, as shown in Figure P7.21.

Figure P7.21 Balance Summary for Customers Who Made Purchases

Minimum Balance	Maximum Balance	Average Balance
0	345.86	112.48

22. Create a query to find the balance characteristics for all customers, including the total of the outstanding balances. The results of this query are shown in Figure P7.22.

Figure P7.22 Balance Summary for All Customers

Total Balances	Minimum Balance	Maximum Balance	Average Balance
2089.28	0.00	768.93	208.93

23. Find the listing of customers who did not make purchases during the invoicing period. Sort the results by customer code. Your output must match the output shown in Figure P7.23.

Figure P7.23 Balances of Customers Who Did Not Make Purchases

CUS_CODE	CUS_BALANCE
10010	0.00
10013	536.75
10016	221.19
10017	768.93
10019	0.00

24. Find the customer balance summary for all customers who have not made purchases during the current invoicing period. The results are shown in Figure P7.24.

Figure P7.24 Summary of Customer Balances for Customers Who Did Not Make Purchases

Total Balance	Minimum Balance	Maximum Balance	Average Balance
1526.87	0.00	768.93	305.37

25. Create a query that summarizes the value of products currently in inventory. Note that the value of each product is a result of multiplying the units currently in inventory by the unit price. Sort the results in descending order by subtotal, as shown in Figure P7.25.

Figure P7.25 Value of Products Currently in Inventory

P_DESCRIPT	P_QOH	P_PRICE	Subtotal
Hicut chain saw, 16 in.	11	256.99	2826.89
Steel matting, 4'x8'x1/6", .5" mesh	18	119.95	2159.10
2.5-in. wd. screw, 50	237	8.45	2002.65
1.25-in. metal screw, 25	172	6.99	1202.28
PVC pipe, 3.5-in., 8-ft	188	5.87	1103.56
Hrd. cloth, 1/2-in., 3x50	23	43.99	1011.77
Power painter, 15 psi., 3-nozzle	8	109.99	879.92
B&D jigsaw, 12-in. blade	8	109.92	879.36
Hrd. cloth, 1/4-in., 2x50	15	39.95	599.25
B&D jigsaw, 8-in. blade	6	99.87	599.22
7.25-in. pwr. saw blade	32	14.99	479.68
B&D cordless drill, 1/2-in.	12	38.95	467.40
9.00-in. pwr. saw blade	18	17.49	314.82
Claw hammer	23	9.95	228.85
Rat-tail file, 1/8-in. fine	43	4.99	214.57
Sledge hammer, 12 lb.	8	14.40	115.20

26. Find the total value of the product inventory. The results are shown in Figure P7.26.

Figure P7.26 Total Value of All Products in Inventory

Total Value of Inventory
15084.52